

BEE 4750 Homework 5: Mixed Integer and Stochastic Programming

Name: Sarah Vafiadis, Nathan Rhoads

ID: sv439

Due Date

Thursday, 12/04/24, 9:00pm

Overview

Instructions

- In Problem 1, you will use mixed integer programming to solve a waste load allocation problem.
- In Problem 2, you will formulate a stochastic optimization problem.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

```
Activating project at `c:\Users\sarah\OneDrive\Documents\GitHub\hw5-sarah_nathan_hw5`
```

```

using JuMP
using HiGHS
using DataFrames
using GraphRecipes
using Plots
using Measures
using MarkdownTables

```

Problems (Total: 30 Points)

Problem 1 (24 points)

Three cities are developing a coordinated municipal solid waste (MSW) disposal plan. Three disposal alternatives are being considered: a landfill (LF), a materials recycling facility (MRF), and a waste-to-energy facility (WTE). The capacities of these facilities and the fees for operation and disposal are provided below.

- **LF:** Capacity 200 Mg, fixed cost \$2000/day, tipping cost \$50/Mg;
- **MRF:** Capacity 350 Mg, fixed cost \$1500/day, tipping cost \$7/Mg, recycling cost \$40/Mg recycled;
- **WTE:** Capacity 210 Mg, fixed cost \$2500/day, tipping cost \$60/Mg;

The MRF recycling rate is 40%, and the ash fraction of non-recycled waste is 16% and of recycled waste is 14%. Transportation costs are \$1.5/Mg-km, and the relative distances between the cities and facilities are provided in the table below.

City/Facility	Landfill (km)	MRF (km)	WTE (km)
1	5	30	15
2	15	25	10
3	13	45	20
LF	-	32	18
MRF	32	-	15
WTE	18	15	-

The fixed costs associated with the disposal options are incurred only if the particular disposal option is implemented. The three cities produce 100, 90, and 120 Mg/day of solid waste, respectively, with the composition provided in the table below.

Component	% of total mass	Combustion ash (%)	MRF Recycling rate (%)
Food Wastes	15	8	0
Paper & Cardboard	40	7	55
Plastics	5	5	15
Textiles	3	10	10
Rubber, Leather	2	15	0
Wood	5	2	30
Yard Wastes	18	2	40
Glass	4	100	60
Ferrous	2	100	75
Aluminum	2	100	80
Other Metal	1	100	50
Miscellaneous	3	70	0

The information in the above table will help you determine the overall recycling and ash fractions. Note that the recycling residuals, which may be sent to either landfill or the WTE, have different ash content than the ash content of the original MSW. You will need to determine these fractions to construct your mass balance constraints.

Reminder: Use `round(x; digits=n)` to report values to the appropriate precision!

Problem 1.1

Based on the information above, calculate the overall recycling and ash fractions for the waste produced by each city.

```
# Component table (mass share, ash %, MRF recycle %)
comp = [
    (0.15, 0.08, 0.00),    # Food waste
    (0.40, 0.07, 0.55),    # Paper and cardboard
    (0.05, 0.05, 0.15),    # Plastics
    (0.03, 0.10, 0.10),    # Textiles
    (0.02, 0.15, 0.00),    # Rubber, leather
    (0.05, 0.02, 0.30),    # Wood
    (0.18, 0.02, 0.40),    # Yard wastes
    (0.04, 1.00, 0.60),    # Glass
    (0.02, 1.00, 0.75),    # Ferrous
    (0.02, 1.00, 0.80),    # Aluminum
    (0.01, 1.00, 0.50),    # Other metals
    (0.03, 0.70, 0.00)     # Misc
```

```

]

# Calculate fractions
recycle = sum(x[1] * x[3] for x in values(comp))
ash = sum(x[1] * x[2] for x in values(comp))

println("Recycling fraction: ", round(recycle, digits=3))
println("Ash fraction: ", round(ash, digits=3))

```

Recycling fraction: 0.378

Ash fraction: 0.164

Problem 1.2

What are the decision variables for your optimization problem? Provide notation and variable meaning.

Decision variables

W_{ij} Waste transported from source i to disposal j (mg/day)

R_{kj} Residual waste transported from disposal k to disposal j (mg/day)

Y_j Operational status of disposal j (binary); 0 if facility not operated and 1 if facility operated

Additional variables

i = cities 1,2,3

j = LF, MRF, WTE

Problem 1.3

Formulate the objective function. Make sure to include any needed derivations or justifications for your equation(s).

minimize $2000Y_{LF} + 1500Y_{MRF} + 2500Y_{WTE}$

FINISH THIS

Problem 1.5

Find the optimal solution (using JuMP to solve the problem). Report the optimal objective value.

```
# Data

facilities = [:LF, :MRF, :WTE]

# City waste generation (mg/d)
cities = 1:3
waste = Dict{1 => 100.0, 2 => 90.0, 3 => 120.0}

# Distances from facilities to cities (km)
dist_city = Dict(
    :LF => Dict{1 => 5.0, 2 => 15.0, 3 => 13.0},
    :MRF => Dict{1 => 30.0, 2 => 25.0, 3 => 45.0},
    :WTE => Dict{1 => 15.0, 2 => 10.0, 3 => 20.0}
)

# Distances between facilities (km)
dist_fac = Dict(
    (:MRF, :LF) => 32.0,
    (:MRF, :WTE) => 15.0,
    (:WTE, :LF) => 18.0
)

# Capacities (Mg/day)
cap = Dict{:LF => 200.0, :MRF => 350.0, :WTE => 210.0}

# Fixed costs ($/day)
F = Dict{:LF => 2000.0, :MRF => 1500.0, :WTE => 2500.0}

# Tipping costs ($/mg)
tipping = Dict{:LF => 50.0, :MRF => 7.0, :WTE => 60.0}

transport = 1.5      # Transportation costs ($/mg-km)
recycle = 40.0       # Recycling costs ($/mg recycled)

MRF_recycle = 0.4    # MRF recycling rate
ash_NR = 0.16        # Ash fraction of non-recycled waste
ash_recycle = 0.14   # Ash fraction of recycled waste
```

```

# Composition table (mass share, ash %, MRF recycle %)
# Component => (mass_share, ash_frac, recycle_frac)
comp = Dict(
    :Food          => (0.15, 0.08, 0.00),
    :PaperCardboard => (0.40, 0.07, 0.55),
    :Plastics       => (0.05, 0.05, 0.15),
    :Textiles       => (0.03, 0.10, 0.10),
    :RubberLeather  => (0.02, 0.15, 0.00),
    :Wood           => (0.05, 0.02, 0.30),
    :Yard           => (0.18, 0.02, 0.40),
    :Glass          => (0.04, 1.00, 0.60),
    :Ferrous        => (0.02, 1.00, 0.75),
    :Aluminum       => (0.02, 1.00, 0.80),
    :OtherMetal     => (0.01, 1.00, 0.50),
    :Misc           => (0.03, 0.70, 0.00)
)

#FIX ALL OF THIS IT MIGHT BE WRONG

# Model
model = Model(HiGHS.Optimizer)

@variable(model, x[cities, facilities] >= 0)      # city-to-facility flows
@variable(model, r[cities] >= 0)                  # recycled mass from MRF
@variable(model, u[cities] >= 0)                  # residual mass from MRF

# Inter-facility flows (FIXED: separate variables or use pairs as indices)
@variable(model, y_mrf_lf >= 0)                   # MRF residuals to LF
@variable(model, y_mrf_wte >= 0)                   # MRF residuals to WTE
@variable(model, y_wte_lf >= 0)                   # WTE ash to LF

@variable(model, z[facilities], Bin)              # open/closed decisions

@constraint(model, [i in cities],
    sum(x[i, f] for f in facilities) == waste[i]
)

# 1. Overall MRF Recycling Rate (MRF_recycle)
# Sum of (mass_share * recycle_frac) for all components
MRF_recycle_calc = sum(c[1] * c[3] for c in values(comp))

# 2. Ash Fraction of Raw Waste (ash_raw)

```

```

# Sum of (mass_share * ash_frac) for all components
ash_raw_calc = sum(c[1] * c[2] for c in values(comp))

# 3. Ash Fraction of MRF Residuals (ash_resid)
# This applies to the non-recycled portion (u[i])
# Numerator: Sum of mass * (1 - recycle_rate) * ash_frac
# Denominator: Sum of mass * (1 - recycle_rate)
numer = sum(c[1] * (1 - c[3]) * c[2] for c in values(comp))
denom = sum(c[1] * (1 - c[3]) for c in values(comp))
ash_resid_calc = numer / denom

# Overwrite the initial placeholders with calculated values
MRF_recycle = MRF_recycle_calc
ash_raw = ash_raw_calc # Use this for direct-to-WTE waste (x[i, :WTE])
ash_resid = ash_resid_calc # Use this for MRF residuals sent to WTE (y[:MRF, :WTE])

println("Calculated Parameters:")
println(" MRF Recycling Rate: ", round(MRF_recycle, digits=4))
println(" Ash (Raw Waste): ", round(ash_raw, digits=4))
println(" Ash (MRF Resid): ", round(ash_resid, digits=4))

```

Problem 1.6

Draw a diagram showing the flows of waste between the cities and the facilities. Which facilities (if any) will not be used? Does this solution make sense?

Problem 2 (6 points)

Consider a two-period economic dispatch problem, based on the multi-period example from Lecture 14 (on 10/29). The generator data, including ramping constraints for each generator, is provided in 'data/generators.csv.' In period 1, the demand is $d_1 = 1100\text{MW}$. In period 2, the demand is projected to be $d_2 = 1200\text{MW}$, but there is a 25% probability that it is \$1500. In the first period, the solar capacity factor is 0.9 and the wind capacity factor is 0.45, but in the second period, there is some uncertainty: the forecasted solar and wind capacity factors are 0.95 and 0.4, respectively, but there is a 30% probability that they are 0.75 and 0.5. Your goal is to identify how to dispatch your generators to minimize the cost of meeting demand.

Problem 2.1

Draw a scenario tree for this problem.

Problem 2.2

Formulate a stochastic linear program for this problem based on your scenario tree from Problem 2.1 and the data in `data/generators.csv`.

```
using JuMP
using HiGHS

G = ["Biomass", "Hydro", "Geo", "NG_CCGT", "NG_CT", "Wind", "Solar"] #G is the different generation

Pmin = Dict("Biomass"=>0, "Hydro"=>0, "Geo"=>0, "NG_CCGT"=>100, "NG_CT"=>100, "Wind"=>0, "Solar"=>0)
Pmax = Dict("Biomass"=>100, "Hydro"=>500, "Geo"=>400, "NG_CCGT"=>250, "NG_CT"=>250, "Wind"=>300, "Solar"=>0)
Cost = Dict("Biomass"=>5, "Hydro"=>0, "Geo"=>0, "NG_CCGT"=>23, "NG_CT"=>38, "Wind"=>0, "Solar"=>0)
Ramp = Dict("Biomass"=>100, "Hydro"=>500, "Geo"=>400, "NG_CCGT"=>100, "NG_CT"=>200, "Wind"=>300, "Solar"=>0)

d1 = 1100.0 #demand in period 1

#different scenarios for period 2 from the tree, including (name, demand, solar_cf, wind_cf,
scenarios = [
    ("S1", 1200.0, 0.95, 0.40, 0.525),
    ("S2", 1200.0, 0.75, 0.50, 0.225),
    ("S3", 1500.0, 0.95, 0.40, 0.175),
    ("S4", 1500.0, 0.75, 0.50, 0.075)
]

#renewable output (fixed)
solar1 = Pmax["Solar"] * 0.90 #500 * .9 = 450
wind1 = Pmax["Wind"] * 0.45 #300 * .45 = 135

m = Model(HiGHS.Optimizer) #write the optimization model

#p1(g) = generation in period 1 MWh
```



```

@variable(m, p1[g in G], lower_bound=Pmin[g], upper_bound=Pmax[g])

#fix renewables p1 so they are not decision variables
fix(p1["Solar"], solar1; force=true) #help from stackoverflow, still unsure what this does
fix(p1["Wind"], wind1; force=true)

#p2(g,s)
@variable(m, p2[g in G, s in 1:4], lower_bound=Pmin[g], upper_bound=Pmax[g])

for s in 1:length(scenarios)
    scenario = scenarios[s]

    solar_cf = scenario[3] #third
    wind_cf = scenario[4] #fourth

    fix(p2["Solar", s], Pmax["Solar"] * solar_cf; force=true)
    fix(p2["Wind", s], Pmax["Wind"] * wind_cf; force=true)
end

#p1
@constraint(m, sum(p1[g] for g in G) == d1)

#p2 (one per scenario)
for s in 1:length(scenarios)
    d2 = scenarios[s][2] #second
    @constraint(m, sum(p2[g, s] for g in G) == d2)
end
#ramping constraints
for g in G
    if g != "Solar" && g != "Wind"
        RU = Ramp[g]
        for s in 1:length(scenarios)
            @constraint(m, p2[g,s] - p1[g] <= RU)
            @constraint(m, p1[g] - p2[g,s] <= RU)
        end
    end
end

```

```

    end
end
#objective
@objective(m, Min,
    sum(Cost[g] * p1[g] for g in G) +
    sum(
        scenarios[s][5] * Cost[g] * p2[g,s]
        for s in 1:length(scenarios), g in G
    )
)
#optimize
optimize!(m)
#start printing the results
println("Expected cost = ", objective_value(m))

println("\np1:")
for g in G
    println(g, ": ", value(p1[g]))
end

println("\np2 by scenario:")
for s in 1:length(scenarios)
    name = scenarios[s][1]
    println(name)
    for g in G
        println("  ", g, ": ", value(p2[g,s]))
    end
end
end

```

Running HiGHS 1.12.0 (git hash: 755a8e027): Copyright (c) 2025 HiGHS under MIT licence terms
LP has 45 rows; 35 cols; 115 nonzeros
Coefficient ranges:
Matrix [1e+00, 1e+00]
Cost [4e-01, 4e+01]
Bound [1e+02, 5e+02]
RHS [1e+02, 2e+03]
Presolving model
13 rows, 19 cols, 35 nonzeros 0s

6 rows, 11 cols, 16 nonzeros 0s
4 rows, 5 cols, 8 nonzeros 0s
Presolve reductions: rows 4(-41); columns 5(-30); nonzeros 8(-107)
Solving the presolved LP

Performed postsolve
Solving the original LP from the solution after postsolve

Model status : Optimal
Objective value : 1.2200000000e+04
P-D objective error : 0.0000000000e+00
HiGHS run time : 0.00
Expected cost = 12200.0

p1:
Biomass: 0.0
Hydro: 315.0
Geo: 0.0
NG_CCGT: 100.0
NG_CT: 100.0
Wind: 135.0
Solar: 450.0

p2 by scenario:

S1

Biomass: 0.0
Hydro: 405.0
Geo: 0.0
NG_CCGT: 100.0
NG_CT: 100.0
Wind: 120.0
Solar: 475.0

S2

Biomass: 0.0
Hydro: 475.0
Geo: 0.0
NG_CCGT: 100.0
NG_CT: 100.0
Wind: 150.0
Solar: 375.0

S3

Biomass: 0.0
Hydro: 305.0

Geo: 400.0
NG_CCGT: 100.0
NG_CT: 100.0
Wind: 120.0
Solar: 475.0
S4
Biomass: 0.0
Hydro: 375.0
Geo: 400.0
NG_CCGT: 100.0
NG_CT: 100.0
Wind: 150.0
Solar: 375.0

References

List any external references consulted, including classmates.